山东大学 计算机科学与技术 学院

自然语言处理 课程实验报告

学号：202200130077	姓名： 薛润泽	班级： 智能 22

实验题目： 实验六：神经机器翻译

实验内容：

1. 用 TensorFlow/pytorch 框架实现 seq2seq 神经机器翻译模型
2. 加入 attention 机制并理解原理(选做）
3. 用 blue 评测翻译结果

实验过程中遇到和解决的问题：

（记录实验过程中遇到的问题，以及解决过程和实验结果。可以适当配以关键代码辅助说明，但不要大段贴代码。）

遇到的问题：实验中没有提供 test 测试集

解决措施：不适用 valid.txt 进行调参，使用 valid.txt 作为测试集

实验步骤：

1、实验原理讲解：

本次实验要求使用 seq2seq 模型进行机器翻译。Seq2seq 模型是一个能够将输入序列转换成另一个输出序列的深度学习模型，输入序列和输出序列长度可以不同。Seq2seq 模型学习的核心目标是学习一个从输入序列到输出序列的映射关系。

Seq2seq 模型主要由两个核心构件组成：编码器 encoder 和解码器 decoder。Encoder 和 decoder 往往使用 RNN, LSTM, GRU 等模型来构建。

编码器的作用是读取并理解输入序列，将输入序列压缩为一个固定长度的上下文向量。这个固定长度的上下文向量捕捉存储了输入序列的全部语义信息。

解码器的作用是接收编码器生成的上下文向量，并基于这个向量逐步生成输出序列。

Seq2Seq 模型的训练一般使用端到端的监督学习方式。

实验要求中提到运用 attention 机制是改进解码器，将 attention 机制运用到解码器中，允许解码器在生成输出序列的每个时间步动态关注输入序列的不同部分，解码器不再仅仅依赖于一个固定的上下文向量，而是可以访问编码器的所有隐藏状态。

2、Encoder 层的构建

创建一个词嵌入层，它会学习一个大小为 (input_dim, emb_dim) 的查找表,当输入一个整数索引（代表一个词）时，它会返回该索引对应的 emb_dim 维的向量。创建 GRU 层作为循环神经网络层。并在 GRU 层后接一个全连接层。同时创建一个 Dropout 层，防止模型过拟合。

```
class Encoder(nn.Module):
    def __init__(self, input_dim, emb_dim, enc_hid_dim, dec_hid_dim, dropout_p):
        super().__init__()
        self.embedding = nn.Embedding(input_dim, emb_dim)
        self.rnn = nn.GRU(emb_dim, enc_hid_dim, bidirectional=True, batch_first=True)
        self.fc = nn.Linear(enc_hid_dim * 2, dec_hid_dim)
        self.dropout = nn.Dropout(dropout_p)

    def forward(self, src):
        embedded = self.dropout(self.embedding(src))
        outputs, hidden = self.rnn(embedded)
        hidden_concat = torch.cat((hidden[-2, :, :], hidden[-1, :, :]), dim=1)
        hidden_transformed = torch.tanh(self.fc(hidden_concat))
        return outputs, hidden_transformed
```

3、Attention 层与 Decoder 层的创建

Decoder 层结合了 attention 机制，forward 前向传播过程中计算了注意力权重和加权的编码器输出，通过 RNN 层进行处理，再经过全连接神经网络输出层进行输出。

```
class Attention(nn.Module):
    def __init__(self, enc_hid_dim, dec_hid_dim):
        super().__init__()
        self.attn = nn.Linear((enc_hid_dim * 2) + dec_hid_dim, dec_hid_dim)
        self.v = nn.Linear(dec_hid_dim, 1, bias=False)

    def forward(self, decoder_hidden, encoder_outputs, src_mask=None):
        batch_size = encoder_outputs.shape[0]
        src_len = encoder_outputs.shape[1]
        decoder_hidden_expanded = decoder_hidden.unsqueeze(1).repeat(1, src_len, 1)
        energy_input = torch.cat((decoder_hidden_expanded, encoder_outputs), dim=2)
        energy = torch.tanh(self.attn(energy_input))
        attention_scores = self.v(energy).squeeze(2)
        if src_mask is not None:
            attention_scores = attention_scores.masked_fill(src_mask == 0, -1e10)
        return F.softmax(attention_scores, dim=1)
```

```
class Decoder(nn.Module):
    def __init__(self, output_dim, emb_dim, enc_hid_dim, dec_hid_dim, dropout_p, attention_module):
        super().__init__()
        self.output_dim = output_dim
        self.attention = attention_module
        self.embedding = nn.Embedding(output_dim, emb_dim)
        self.rnn = nn.GRU((enc_hid_dim * 2) + emb_dim, dec_hid_dim, batch_first=True)
        self.fc_out = nn.Linear((enc_hid_dim * 2) + dec_hid_dim + emb_dim, output_dim)
        self.dropout = nn.Dropout(dropout_p)

    def forward(self, trg_input, decoder_hidden, encoder_outputs, src_mask=None):
        trg_input_unsqueezed = trg_input.unsqueeze(1)
        embedded = self.dropout(self.embedding(trg_input_unsqueezed))
        attention_weights = self.attention(decoder_hidden, encoder_outputs, src_mask).unsqueeze(1)
        weighted_encoder_output = torch.bmm(attention_weights, encoder_outputs)
        rnn_input = torch.cat((embedded, weighted_encoder_output), dim=2)
        decoder_hidden_rnn_input = decoder_hidden.unsqueeze(0)
        rnn_output, new_decoder_hidden = self.rnn(rnn_input, decoder_hidden_rnn_input)
        embedded_squeezed = embedded.squeeze(1)
        weighted_encoder_output_squeezed = weighted_encoder_output.squeeze(1)
        rnn_output_squeezed = rnn_output.squeeze(1)
        fc_input = torch.cat((rnn_output_squeezed, weighted_encoder_output_squeezed, embedded_squeezed), dim=1)
        prediction = self.fc_out(fc_input)
        return prediction, new_decoder_hidden.squeeze(0)
```

4、Seq2seq 层的组合创建

```python
class Seq2Seq(nn.Module):
    def __init__(self, encoder, decoder, src_pad_idx, device):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.src_pad_idx = src_pad_idx
        self.device = device
    def create_mask(self, src):
        mask = (src != self.src_pad_idx)
        return mask
    def forward(self, src, trg, teacher_forcing_ratio=0.5):
        batch_size = trg.shape[0]
        trg_len = trg.shape[1]
        trg_vocab_size = self.decoder.output_dim
        outputs = torch.zeros(batch_size, trg_len, trg_vocab_size).to(self.device)
        src_mask = self.create_mask(src)
        encoder_outputs, decoder_hidden = self.encoder(src)
        current_trg_input = trg[:, 0]
        for t in range(1, trg_len):
            decoder_output, decoder_hidden = self.decoder(current_trg_input, decoder_hidden, encoder_outputs, src_mask)
            outputs[:, t] = decoder_output
            use_teacher_force = random.random() < teacher_forcing_ratio
            if use_teacher_force:
                current_trg_input = trg[:, t]
            else:
                top1_prediction = decoder_output.argmax(1)
                current_trg_input = top1_prediction
        return outputs
```

5、BlEU 评测函数

BlEU 评测函数计算给定数据集上神经机器翻译模型生成的翻译结果的 BLEU 分数。BLEU 分数通过比较机器翻译的输出与一个或者多个专业人工翻译的参考句之间的 n-gram 重叠程度来衡量翻译的流畅度和准确度。

```python
def calculate_bleu_score(dataset, model, src_tokenizer, src_vocab, trg_vocab, device, max_len=50):
    targets_tokenized = []
    predictions_tokenized = []
    model.eval()
    for i in range(len(dataset)):
        src_text_raw = dataset.raw_src_texts[i]
        trg_tokens_raw = dataset.raw_trg_texts_tokenized[i]
        prediction_tokens = translate_sentence_new(
            src_text_raw,
            src_tokenizer,
            src_vocab,
            trg_vocab,
            model,
            device,
            max_len
        )

        predictions_tokenized.append(prediction_tokens)
        targets_tokenized.append([trg_tokens_raw])

        if i < 5:
            print(f"Source: {src_text_raw}")
            print(f"Target: {' '.join(trg_tokens_raw)}")
            print(f"Predicted: {' '.join(prediction_tokens)}\n")
    chencherry = SmoothingFunction()
    bleu = corpus_bleu(targets_tokenized, predictions_tokenized, smoothing_function=chencherry.method4)
    return bleu * 100
```

实验结果：

分别执行训练轮次 10、20、30 次，使用 valid.txt 作为测试集，测试 BLEU 结果分数如下

训练 10 轮测试

```
Calculating BLEU score on the test set...
Source: The function of a customer is defined by assigning an account group.
Target: 通过 分配 一个 账户 组来 定义 一个 客户 的 功能 。
Predicted: 一个 客户 的 提交 是 通过 一个 方 的 的 格言 。

Source: I argued a book's assigning us to read is no good.
Target: 你 说 教授 指定 要 我们 看 的 书 不好 ！ 你 真是 没有 分寸 . 嗯 ， 你 刚才 还 说 什么 来 啦 ？
Predicted: 我 本书 本书 是 我们 我们 的 的 。

Source: A brave fighter never shrinks from danger in executing an assignment.
Target: 一个 英勇 的 战士 在 执行 任务 时 决不 害怕 危险 。
Predicted: 一个 健美 的 没有 在 危险 任务 中 没有 危险 的 。

Source: He is a yellow-bellied guy; don't send him on such a tough assignment!
Target: 他 是 个 胆小 的 ， 不要 给 他 这么 艰难 的 工作 。
Predicted: 他 是 个 个 的 棒球 的 人 ， 没 把 他 的 他 的 的 任务 。

Source: She's going to India on a special assignment for her newspaper.
Target: 她 受 委派 到 印度 去 执行 报社 的 一项 特殊任务 。
Predicted: 她 对 她 的 的 的 特别 专门 为 特别 的 特别 。

Corpus BLEU score on Test Set: 3.46
```

训练 20 轮测试

```
Calculating BLEU score on the test set...
Source: The function of a customer is defined by assigning an account group.
Target: 通过 分配 一个 账户 组来 定义 一个 客户 的 功能 。
Predicted: 客户 的 核心 是 通过 对 分开 的 内容 来 定义 。

Source: I argued a book's assigning us to read is no good.
Target: 你 说 教授 指定 要 我们 看 的 书 不好 ！ 你 真是 没有 分寸 . 嗯 ， 你 刚才 还 说 什么 来 啦 ？
Predicted: 我 本书 了 本书 本书 告诉 我们 并 不 。

Source: A brave fighter never shrinks from danger in executing an assignment.
Target: 一个 英勇 的 战士 在 执行 任务 时 决不 害怕 危险 。
Predicted: 一个 健美 的 击球手 永远 不会 在 两次 执行 任务 。

Source: He is a yellow-bellied guy; don't send him on such a tough assignment!
Target: 他 是 个 胆小 的 ， 不要 给 他 这么 艰难 的 工作 。
Predicted: 他 是 个 普通 的 棒球 的 人 --- 不要 把 他 放在 一个 一个 的 任务 ！

Source: She's going to India on a special assignment for her newspaper.
Target: 她 受 委派 到 印度 去 执行 报社 的 一项 特殊任务 。
Predicted: 她 在 她 的 面前 ， 她 对 她 来说 特别 特殊 的 特别 。

Corpus BLEU score on Test Set: 3.50
```

训练 30 轮测试

```
Calculating BLEU score on the test set...
Source: The function of a customer is defined by assigning an account group.
Target: 通过 分配 一个 账户 组来 定义 一个 客户 的 功能 。
Predicted: 客户 的 客户 的 通过 确定 方 发起 的 的 的 定义 。

Source: I argued a book's assigning us to read is no good.
Target: 你 说 教授 指定 要 我们 看 的 书 不好 ！ 你 真是 没有 分寸 . 嗯 ， 你 刚才 还 说 什么 来 啦 ？
Predicted: 我 认为 那本书 是 我们 的 买卖 ， 并 不是 。

Source: A brave fighter never shrinks from danger in executing an assignment.
Target: 一个 英勇 的 战士 在 执行 任务 时 决不 害怕 危险 。
Predicted: 一个 健美 的 击球手 从来 没有 成功 过 危险 的 任务 。

Source: He is a yellow-bellied guy; don't send him on such a tough assignment!
Target: 他 是 个 胆小 的 ， 不要 给 他 这么 艰难 的 工作 。
Predicted: 他 是 个 低沉 的 棒球 家伙 ， 可是 不要 把 他 给 他 的 成功 的 任务 ！

Source: She's going to India on a special assignment for her newspaper.
Target: 她 受 委派 到 印度 去 执行 报社 的 一项 特殊任务 。
Predicted: 她 在 她 的 的 来说 ， 专门 为 专门 的 特别 的 。

Corpus BLEU score on Test Set: 3.72
```

实验总结：
可以看到，随着训练轮次增加，机器翻译效果逐渐变好，BLEU 评测分数逐渐增加

附源代码

整体项目由 main.py，constrains.py, train.py, model.py, data_process.py 等多个文件组合而成

Main.py

```
import os
import math
import time
import torch
import torch.nn as nn
import torch.optim as optim
import random
import spacy
import jieba
import csv
import argparse
import re

from nltk.translate.bleu_score import corpus_bleu, SmoothingFunction

from torch.utils.data import Dataset, DataLoader
from torchtext.vocab import build_vocab_from_iterator
from torch.nn.utils.rnn import pad_sequence

from constrains import (N_EPOCHS, ENC_EMB_DIM, ENC_HID_DIM, DEC_HID_DIM,
                        ENC_DROPOUT, DEC_EMB_DIM, DEC_DROPOUT, device,
SEED,
                        ORIGINAL_TRAIN_FILE, PROCESSED_TRAIN_FILE,
ORIGINAL_VALID_FILE, PROCESSED_VALID_FILE,
                        ORIGINAL_TEST_FILE, PROCESSED_TEST_FILE,
BATCH_SIZE, CLIP
                        )
from train import train, evaluate, epoch_time
from model import Seq2Seq, Attention, Encoder, Decoder
from data_process import (preprocess_to_tsv, load_sentences)

parser = argparse.ArgumentParser()
```

```
parser.add_argument('-m', '--model_path',type=str, default=None)
parser.add_argument('-t','--test_model', type=str, default=None)
args = parser.parse_args()

os.environ["TMP"] = r"temp"
os.environ["TEMP"] = r"temp"

random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed(SEED)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

print(f"Using device: {device}")

print('Data process: ')
preprocess_to_tsv(ORIGINAL_TRAIN_FILE, PROCESSED_TRAIN_FILE)
preprocess_to_tsv(ORIGINAL_TEST_FILE, PROCESSED_TEST_FILE)

spacy_en = spacy.load('en_core_web_sm')

def tokenize_en(text):
    return [tok.text.lower() for tok in spacy_en.tokenizer(text)][::-1]

def tokenize_zh(text):
    return [tok.lower() for tok in jieba.cut(text)]

UNK_TOKEN, PAD_TOKEN, SOS_TOKEN, EOS_TOKEN = '<unk>', '<pad>', '<sos>',
'<eos>'
special_symbols = [UNK_TOKEN, PAD_TOKEN, SOS_TOKEN, EOS_TOKEN]

def yield_tokens_from_tsv(file_path, tokenizer_fn, column_index):
    with open(file_path, 'r', encoding='utf-8') as f:
        reader = csv.reader(f, delimiter='\t')
        for row in reader:
            if row:
                yield tokenizer_fn(row[column_index])

print("Building source vocabulary...")
SRC_VOCAB = build_vocab_from_iterator(
    yield_tokens_from_tsv(PROCESSED_TRAIN_FILE, tokenize_en, 0),
    min_freq=1,
    specials=special_symbols,
```

```
    special_first=True
)
SRC_VOCAB.set_default_index(SRC_VOCAB[UNK_TOKEN])
SRC_PAD_IDX = SRC_VOCAB[PAD_TOKEN]
SRC_SOS_IDX = SRC_VOCAB[SOS_TOKEN]
SRC_EOS_IDX = SRC_VOCAB[EOS_TOKEN]

print("Building target vocabulary...")
TRG_VOCAB = build_vocab_from_iterator(
    yield_tokens_from_tsv(PROCESSED_TRAIN_FILE, tokenize_zh, 1),
    min_freq=1,
    specials=special_symbols,
    special_first=True
)
TRG_VOCAB.set_default_index(TRG_VOCAB[UNK_TOKEN])
TRG_PAD_IDX = TRG_VOCAB[PAD_TOKEN]
TRG_SOS_IDX = TRG_VOCAB[SOS_TOKEN]
TRG_EOS_IDX = TRG_VOCAB[EOS_TOKEN]

print(f"Unique tokens in source (en) vocabulary: {len(SRC_VOCAB)}")
print(f"Unique tokens in target (zh) vocabulary: {len(TRG_VOCAB)}")

class TranslationDataset(Dataset):
    def __init__(self, tsv_file_path, src_tokenizer, trg_tokenizer,
src_vocab, trg_vocab):
        self.src_tokenizer = src_tokenizer
        self.trg_tokenizer = trg_tokenizer
        self.src_vocab = src_vocab
        self.trg_vocab = trg_vocab
        self.data = []
        self.raw_src_texts = []
        self.raw_trg_texts_tokenized = []

        with open(tsv_file_path, 'r', encoding='utf-8') as f:
            reader = csv.reader(f, delimiter='\t')
            for row in reader:
                if len(row) == 2:
                    src_text_raw, trg_text_raw = row[0], row[1]
                    self.data.append((src_text_raw, trg_text_raw))
                    self.raw_src_texts.append(src_text_raw)
                    self.raw_trg_texts_tokenized.append(self.trg_tokeniz
er(trg_text_raw))


    def __len__(self):
```

```
        return len(self.data)

    def __getitem__(self, idx):
        src_text, trg_text = self.data[idx]
        src_tokens_numericalized = [SRC_SOS_IDX] + [self.src_vocab[token]
for token in self.src_tokenizer(src_text)] + [SRC_EOS_IDX]
        trg_tokens_numericalized = [TRG_SOS_IDX] + [self.trg_vocab[token]
for token in self.trg_tokenizer(trg_text)] + [TRG_EOS_IDX]
        return torch.tensor(src_tokens_numericalized, dtype=torch.long),
torch.tensor(trg_tokens_numericalized, dtype=torch.long)

def collate_batch(batch):
    src_list, trg_list = [], []
    for (src_item, trg_item) in batch:
        src_list.append(src_item)
        trg_list.append(trg_item)
    src_padded = pad_sequence(src_list, batch_first=True,
padding_value=SRC_PAD_IDX)
    trg_padded = pad_sequence(trg_list, batch_first=True,
padding_value=TRG_PAD_IDX)
    return src_padded.to(device), trg_padded.to(device)

train_dataset = TranslationDataset(PROCESSED_TRAIN_FILE, tokenize_en,
tokenize_zh, SRC_VOCAB, TRG_VOCAB)
valid_dataset = TranslationDataset(PROCESSED_VALID_FILE, tokenize_en,
tokenize_zh, SRC_VOCAB, TRG_VOCAB)
test_dataset = TranslationDataset(PROCESSED_VALID_FILE, tokenize_en,
tokenize_zh, SRC_VOCAB, TRG_VOCAB)

print(f"Number of training examples: {len(train_dataset)}")
print(f"Number of validation examples: {len(valid_dataset)}")
print(f"Number of testing examples: {len(test_dataset)}")

train_iterator = DataLoader(train_dataset, batch_size=BATCH_SIZE,
                            shuffle=True, collate_fn=collate_batch)
valid_iterator = DataLoader(valid_dataset, batch_size=BATCH_SIZE,
                            shuffle=False, collate_fn=collate_batch)
test_iterator  = DataLoader(test_dataset, batch_size=BATCH_SIZE,
                        shuffle=False, collate_fn=collate_batch)

print("Start model build init... ")
INPUT_DIM = len(SRC_VOCAB)
OUTPUT_DIM = len(TRG_VOCAB)
attn = Attention(ENC_HID_DIM, DEC_HID_DIM)
```

```
enc = Encoder(INPUT_DIM, ENC_EMB_DIM, ENC_HID_DIM, DEC_HID_DIM,
ENC_DROPOUT)
dec = Decoder(OUTPUT_DIM, DEC_EMB_DIM, ENC_HID_DIM, DEC_HID_DIM,
DEC_DROPOUT, attn)
model = Seq2Seq(enc, dec, SRC_PAD_IDX, device).to(device)

def init_weights(m):
    for name, param in m.named_parameters():
        if 'weight' in name:
            nn.init.normal_(param.data, mean=0, std=0.01)
        else:
            nn.init.constant_(param.data, 0)

optimizer = optim.Adam(model.parameters())
criterion = nn.CrossEntropyLoss(ignore_index=TRG_PAD_IDX)

best_valid_loss = float('inf')

if args.model_path:
    print(f"Loading model from: {args.model_path}")
    model.load_state_dict(torch.load(args.model_path,
map_location=device))
    print("Model state_dict loaded successfully.")
    filename = os.path.basename(args.model_path)
    epoch_match = re.search(r'epoch-(\d+)', filename)
    if epoch_match:
        start_epoch = int(epoch_match.group(1))
        print(f"Inferred start epoch from filename: {start_epoch}. Will
continue from epoch {start_epoch + 1}.")
    else:
        print("Could not infer start epoch from filename. Starting from epoch
0 for loaded model.")
        start_epoch = 0
else:
    print("No pre-trained model specified. Initializing weights randomly.")
    model.apply(init_weights)

if args.test_model is None:
    print("Starting training...")
    effective_start_epoch = start_epoch + 1 if args.model_path and
epoch_match else 0
    for epoch_idx in range(effective_start_epoch, N_EPOCHS):
        epoch = epoch_idx + 1
        start_time = time.time()
```

```
        train_loss = train(model, train_iterator, optimizer, criterion,
CLIP)
        end_time = time.time()
        epoch_mins, epoch_secs = epoch_time(start_time, end_time)
        torch.save(model.state_dict(), f'0602-model-epoch-{epoch}.pt')
        print(f"\tModel saved for epoch {epoch}.")
        print(f'Epoch: {epoch:02} | Time: {epoch_mins}m {epoch_secs}s')
        print(f'\tTrain Loss: {train_loss:.3f} | Train PPL:
{math.exp(train_loss):7.3f}')

def translate_sentence_new(sentence_str, src_tokenizer, src_vocab,
trg_vocab, model, device, max_len=50):
    model.eval()
    if isinstance(sentence_str, str):
        tokens = src_tokenizer(sentence_str)
    else:
        tokens = [token.lower() for token in sentence_str]

    src_indexes = [SRC_SOS_IDX] + [src_vocab.get_stoi().get(token,
SRC_VOCAB[UNK_TOKEN]) for token in tokens] + [SRC_EOS_IDX]
    src_tensor = torch.LongTensor(src_indexes).unsqueeze(0).to(device)
    src_mask_translate = (src_tensor != SRC_PAD_IDX).to(device)

    with torch.no_grad():
        encoder_outputs, hidden = model.encoder(src_tensor)

    trg_indexes = [TRG_SOS_IDX]
    for i in range(max_len):
        trg_tensor = torch.LongTensor([trg_indexes[-1]]).to(device)
        with torch.no_grad():
            output, hidden = model.decoder(trg_tensor, hidden,
encoder_outputs, src_mask_translate) # Pass src_mask
        pred_token_idx = output.argmax(1).item()
        trg_indexes.append(pred_token_idx)
        if pred_token_idx == TRG_EOS_IDX:
            break
    trg_tokens = [trg_vocab.get_itos()[i] for i in trg_indexes]
    return trg_tokens[1:-1] if trg_tokens[-1] == EOS_TOKEN else
trg_tokens[1:]

def calculate_bleu_score(dataset, model, src_tokenizer, src_vocab,
trg_vocab, device, max_len=50):
    targets_tokenized = []
    predictions_tokenized = []
```

```
    model.eval()
    for i in range(len(dataset)):
        src_text_raw = dataset.raw_src_texts[i]
        trg_tokens_raw = dataset.raw_trg_texts_tokenized[i]
        prediction_tokens = translate_sentence_new(
            src_text_raw,
            src_tokenizer,
            src_vocab,
            trg_vocab,
            model,
            device,
            max_len
        )

        predictions_tokenized.append(prediction_tokens)
        targets_tokenized.append([trg_tokens_raw])

        if i < 5:
             print(f"Source: {src_text_raw}")
             print(f"Target: {' '.join(trg_tokens_raw)}")
             print(f"Predicted: {' '.join(prediction_tokens)}\n")
    chencherry = SmoothingFunction()
    bleu = corpus_bleu(targets_tokenized, predictions_tokenized,
smoothing_function=chencherry.method4)
    return bleu * 100

if args.test_model is not None:
    print(f"Loading model for testing from: {args.test_model}")
    model.load_state_dict(torch.load(args.test_model,
map_location=device))
    print("Model loaded successfully for testing.")

    test_loss = evaluate(model, test_iterator, criterion)
    print(f'| Test Loss: {test_loss:.3f} | Test PPL:
{math.exp(test_loss):7.3f} |')
    print("\nCalculating BLEU score on the test set...")
    bleu_score_test = calculate_bleu_score(
        test_dataset,
        model,
        tokenize_en,
        SRC_VOCAB,
        TRG_VOCAB,
        device
    )
```

```
    print(f"Corpus BLEU score on Test Set: {bleu_score_test:.2f}")
```

Constrains.py

```
N_EPOCHS = 10
CLIP = 1
best_valid_loss = float('inf')

device = 'cuda'
BATCH_SIZE = 4
SEED = 1234


ENC_EMB_DIM = 128
DEC_EMB_DIM = 128
ENC_HID_DIM = 256
DEC_HID_DIM = 256
ENC_DROPOUT = 0.5
DEC_DROPOUT = 0.5

ORIGINAL_TRAIN_FILE = 'train.txt'
ORIGINAL_VALID_FILE = 'valid.txt'
ORIGINAL_TEST_FILE = 'test.txt'

PROCESSED_TRAIN_FILE = 'train.tsv'
PROCESSED_VALID_FILE = 'valid.tsv'
PROCESSED_TEST_FILE = 'test.tsv'
```

Model.py

```
import torch
import torch.nn as nn
import torch.nn.functional as F # 确保导入
import math
import random # 确保导入

class Encoder(nn.Module):
    def __init__(self, input_dim, emb_dim, enc_hid_dim, dec_hid_dim,
dropout_p):
        super().__init__()
        self.embedding = nn.Embedding(input_dim, emb_dim)
        self.rnn = nn.GRU(emb_dim, enc_hid_dim, bidirectional=True,
batch_first=True)
        self.fc = nn.Linear(enc_hid_dim * 2, dec_hid_dim)
```

```
        self.dropout = nn.Dropout(dropout_p)

    def forward(self, src):
        embedded = self.dropout(self.embedding(src))
        outputs, hidden = self.rnn(embedded)
        hidden_concat = torch.cat((hidden[-2, :, :], hidden[-1, :, :]),
dim=1)
        hidden_transformed = torch.tanh(self.fc(hidden_concat))
        return outputs, hidden_transformed

class Attention(nn.Module):
    def __init__(self, enc_hid_dim, dec_hid_dim):
        super().__init__()
        self.attn = nn.Linear((enc_hid_dim * 2) + dec_hid_dim, dec_hid_dim)
        self.v = nn.Linear(dec_hid_dim, 1, bias=False)

    def forward(self, decoder_hidden, encoder_outputs, src_mask=None):
        batch_size = encoder_outputs.shape[0]
        src_len = encoder_outputs.shape[1]
        decoder_hidden_expanded = decoder_hidden.unsqueeze(1).repeat(1,
src_len, 1)
        energy_input = torch.cat((decoder_hidden_expanded,
encoder_outputs), dim=2)
        energy = torch.tanh(self.attn(energy_input))
        attention_scores = self.v(energy).squeeze(2)
        if src_mask is not None:
            attention_scores = attention_scores.masked_fill(src_mask == 0,
-1e10)
        return F.softmax(attention_scores, dim=1)

class Decoder(nn.Module):
    def __init__(self, output_dim, emb_dim, enc_hid_dim, dec_hid_dim,
dropout_p, attention_module):
        super().__init__()
        self.output_dim = output_dim
        self.attention = attention_module
        self.embedding = nn.Embedding(output_dim, emb_dim)
        self.rnn = nn.GRU((enc_hid_dim * 2) + emb_dim, dec_hid_dim,
batch_first=True)
        self.fc_out = nn.Linear((enc_hid_dim * 2) + dec_hid_dim + emb_dim,
output_dim)
        self.dropout = nn.Dropout(dropout_p)
```

```
    def forward(self, trg_input, decoder_hidden, encoder_outputs,
src_mask=None):
        trg_input_unsqueezed = trg_input.unsqueeze(1)
        embedded = self.dropout(self.embedding(trg_input_unsqueezed))
        attention_weights = self.attention(decoder_hidden,
encoder_outputs, src_mask).unsqueeze(1)
        weighted_encoder_output = torch.bmm(attention_weights,
encoder_outputs)
        rnn_input = torch.cat((embedded, weighted_encoder_output), dim=2)
        decoder_hidden_rnn_input = decoder_hidden.unsqueeze(0)
        rnn_output, new_decoder_hidden = self.rnn(rnn_input,
decoder_hidden_rnn_input)
        embedded_squeezed = embedded.squeeze(1)
        weighted_encoder_output_squeezed =
weighted_encoder_output.squeeze(1)
        rnn_output_squeezed = rnn_output.squeeze(1)
        fc_input = torch.cat((rnn_output_squeezed,
weighted_encoder_output_squeezed, embedded_squeezed), dim=1)
        prediction = self.fc_out(fc_input)
        return prediction, new_decoder_hidden.squeeze(0)

class Seq2Seq(nn.Module):
    def __init__(self, encoder, decoder, src_pad_idx, device):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.src_pad_idx = src_pad_idx
        self.device = device
    def create_mask(self, src):
        mask = (src != self.src_pad_idx)
        return mask
    def forward(self, src, trg, teacher_forcing_ratio=0.5):
        batch_size = trg.shape[0]
        trg_len = trg.shape[1]
        trg_vocab_size = self.decoder.output_dim
        outputs = torch.zeros(batch_size, trg_len,
trg_vocab_size).to(self.device)
        src_mask = self.create_mask(src)
        encoder_outputs, decoder_hidden = self.encoder(src)
        current_trg_input = trg[:, 0]
        for t in range(1, trg_len):
            decoder_output, decoder_hidden =
self.decoder(current_trg_input, decoder_hidden, encoder_outputs,
src_mask)
```

```
            outputs[:, t] = decoder_output
            use_teacher_force = random.random() < teacher_forcing_ratio
            if use_teacher_force:
                current_trg_input = trg[:, t]
            else:
                top1_prediction = decoder_output.argmax(1)
                current_trg_input = top1_prediction
        return outputs
```

Data_process.py

```
def preprocess_to_tsv(original_file_path, processed_file_path):
    print(f"Preprocessing {original_file_path} into
{processed_file_path}...")
    with open(original_file_path, 'r', encoding='utf-8') as infile, \
         open(processed_file_path, 'w', encoding='utf-8') as outfile:
        lines = infile.readlines()
        i = 0
        pairs_written = 0
        while i + 1 < len(lines):
            eng_sentence = lines[i].strip()
            chn_sentence = lines[i+1].strip()
            if eng_sentence and chn_sentence:
                outfile.write(f"{eng_sentence}\t{chn_sentence}\n")
                pairs_written +=1
            i += 2
    return pairs_written > 0
```

Train.py

```
import torch

def train(model, iterator, optimizer, criterion, clip):
    model.train()
    epoch_loss = 0
    for i, batch in enumerate(iterator):
        src, trg = batch
        optimizer.zero_grad()
        output = model(src, trg)
        output_dim = output.shape[-1]
        output = output[:, 1:].reshape(-1, output_dim)
```

```
        trg = trg[:, 1:].reshape(-1)
        loss = criterion(output, trg)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), clip)
        optimizer.step()
        epoch_loss += loss.item()
    return epoch_loss / len(iterator)

def evaluate(model, iterator, criterion):
    model.eval()
    epoch_loss = 0
    with torch.no_grad():
        for i, batch in enumerate(iterator):
            src, trg = batch
            output = model(src, trg, 0)
            output_dim = output.shape[-1]
            output = output[:, 1:].reshape(-1, output_dim)
            trg = trg[:, 1:].reshape(-1)
            loss = criterion(output, trg)
            epoch_loss += loss.item()
    return epoch_loss / len(iterator)

def epoch_time(start_time, end_time):
    elapsed_time = end_time - start_time
    elapsed_mins = int(elapsed_time / 60)
    elapsed_secs = int(elapsed_time - (elapsed_mins * 60))
    return elapsed_mins, elapsed_secs
```